BookBase

Sistema de Gestión Bibliotecaria con GraphQL

Trabajo de la asignatura

Complementos de Bases de Datos

Escuela Técnica Superior de Ingeniería Informática

Universidad de Sevilla

Autores

Juan Ramírez Morales

Ángel Manuel Ferrer Álvarez

Curso académico 2025–2026

Sevilla, abril de 2026

Índice

Índice de tablas iii

Índice de figuras iv

1. Introducción 1
1.1. Contexto y motivación . . . 1
1.2. Objetivos . . . 1
1.3. Metodología . . . 1
1.4. Estructura del documento . . . 2

2. Tecnologías y herramientas 2
2.1. Stack del servidor . . . 2
2.2. Stack del cliente . . . 2
2.3. GraphQL frente a REST . . . 3

3. Diseño del sistema 3
3.1. Arquitectura general . . . 3
3.2. Modelo de datos . . . 3
3.3. Esquema GraphQL y operaciones disponibles . . . 5
3.4. Autenticación y autorización . . . 6

4. Implementación 7
4.1. Backend . . . 7
4.1.1. Punto de entrada y configuración del servidor . . . 7
4.1.2. Organización de resolvers . . . 7
4.1.3. Utilidades del servidor . . . 7
4.1.4. Proceso de préstamo . . . 7
4.2. Frontend . . . 7
4.2.1. Estructura de páginas . . . 7
4.2.2. Panel de administración (`/admin`) . . . 8
4.2.3. Patrón de hooks . . . 8

5. Manual de usuario 9
5.1. Registro e inicio de sesión . . . 9
5.2. Exploración del catálogo . . . 9
5.3. Gestión de préstamos . . . 10
5.4. Gestión de reseñas . . . 11
5.5. Funciones de administrador . . . 11

6. Manual de despliegue 13
6.1. Requisitos previos . . . 13
6.2. Configuración del entorno . . . 13
6.3. Instalación y arranque . . . 13
6.3.1. Backend . . . 13
6.3.2. Cliente web . . . 14
6.4. Aplicación desplegada . . . 14
6.5. Población de datos de prueba . . . 14

7. Conclusiones **15**
7.1. Resumen de resultados 15
7.2. Dificultades y decisiones de diseño 15

Uso de IA **17**

Índice de tablas

1. Stack tecnológico del servidor 2
2. Stack tecnológico del cliente web 3
3. Entidad `Author` 4
4. Entidad `Book` 4
5. Entidad `User` 4
6. Entidad `Loan` 5
7. Entidad `Review` 5
8. Operaciones GraphQL principales 6
9. Utilidades del servidor (`src/utils/`) 7
10. Páginas del cliente React 8
11. Variables de entorno del servidor 13
12. Usuarios de prueba creados por el script de semilla 15

Índice de figuras

1. Catálogo de libros con búsqueda y filtros 9
2. Página de detalle de libro con botón de préstamo 10
3. Vista de préstamos del usuario con opción de devolución 11
4. Vista de reseñas del usuario con opciones de edición y eliminación . . . 11
5. Panel de administración — gestión de libros (`/admin/books`) 12
6. Panel de administración — gestión de autores (`/admin/authors`) . . . 12
7. Panel de administración — vista global de préstamos (`/admin/loans`) . 13

1. Introducción

1.1. Contexto y motivación

La gestión eficiente de recursos bibliográficos es un reto recurrente en instituciones académicas y públicas. Los sistemas tradicionales de bibliotecas, basados en APIs REST o en soluciones monolíticas, presentan limitaciones cuando los clientes necesitan datos de múltiples entidades relacionadas en una sola petición. Esto genera sobrecargas de red (*over-fetching*) o múltiples idas y venidas al servidor (*under-fetching*).

GraphQL (GraphQL Foundation, 2021) es un lenguaje de consulta para APIs desarrollado por Meta que permite a los clientes solicitar exactamente los datos que necesitan, resolviendo ambos problemas. Su adopción ha crecido notablemente en el sector tecnológico desde su publicación como estándar abierto en 2015, y dispone de un ecosistema maduro en el entorno de Node.js, especialmente a través de Apollo Server (Apollo GraphQL, 2023).

Este trabajo presenta el diseño e implementación de **BookBase**, una aplicación web de gestión bibliotecaria que emplea GraphQL como capa de API. El sistema permite a los usuarios explorar un catálogo de libros, gestionar préstamos y publicar reseñas, mientras que los administradores pueden mantener el catálogo actualizado. El resultado es un sistema funcional que demuestra la idoneidad de GraphQL para dominios con datos relacionados, reduciendo la complejidad del cliente y manteniendo un contrato de API explícito y extensible.

1.2. Objetivos

Los objetivos principales del proyecto son:

1. Diseñar e implementar una API GraphQL completa que cubra las operaciones CRUD sobre autores, libros, préstamos y reseñas, con autenticación basada en JWT y control de acceso por roles.
2. Desarrollar un cliente web en React que consuma la API de forma eficiente, ofreciendo una interfaz intuitiva para usuarios finales y administradores.
3. Demostrar la viabilidad de GraphQL con Apollo Server 4 y MongoDB/Mongoose como stack tecnológico para aplicaciones de gestión de datos relacionados.

1.3. Metodología

El desarrollo se llevó a cabo de forma iterativa e incremental. En primer lugar se definió el esquema GraphQL, lo que sirvió de contrato entre el equipo de frontend y backend. A continuación se implementaron los modelos de datos en MongoDB, los resolvers y las utilidades del servidor, y finalmente el cliente React con sus hooks de datos.

Las pruebas se realizaron mediante el Apollo Sandbox, disponible en `http://localhost:4000/graphql`, que permite ejecutar queries y mutations de forma interactiva. El repositorio incluye un fichero `TESTING.md` con el conjunto completo de operaciones de prueba.

Para poblar la base de datos con información real se desarrolló un script de semilla (*seed*) que consume la Open Library API (Internet Archive, 2024), obteniendo aproximadamente 200 obras de informática con sus autores, géneros y descripciones.

1.4. Estructura del documento

El resto del documento se organiza del siguiente modo. El capítulo 2 describe las tecnologías y herramientas utilizadas. El capítulo 3 presenta el diseño del sistema: modelo de datos, esquema GraphQL y mecanismos de seguridad. El capítulo 4 detalla los aspectos más relevantes de la implementación. Los capítulos 5 y 6 constituyen, respectivamente, el manual de usuario y el manual de despliegue. Finalmente, el capítulo 7 recoge las conclusiones del proyecto.

2. Tecnologías y herramientas

2.1. Stack del servidor

La Tabla 1 resume las principales tecnologías empleadas en el servidor.

Tabla 1
Stack tecnológico del servidor

Capa	Tecnología
Entorno de ejecución	Node.js
Lenguaje	TypeScript
Framework API	Apollo Server + Express
Base de datos	MongoDB
ODM	Mongoose
Autenticación	jsonwebtoken / bcryptjs
Validación	Zod
Desarrollo	Nodemon + ts-node

Apollo Server 4 con `expressMiddleware` (en lugar del modo *standalone*) se eligió por su flexibilidad para integrar middlewares Express adicionales y por ser la versión de producción recomendada (Apollo GraphQL, 2023). **MongoDB** con Mongoose (Automattic, Inc., 2024) proporciona un esquema flexible y tipado para los documentos, con soporte nativo para referencias entre colecciones. **TypeScript** en modo estricto garantiza seguridad de tipos en toda la capa de resolvers.

2.2. Stack del cliente

La Tabla 2 resume las tecnologías del cliente web.

Tabla 2
Stack tecnológico del cliente web

Capa	Tecnología
Framework UI	React
Enrutamiento	React Router
Herramienta de build	Vite
Lenguaje	TypeScript
Gestión de estado remoto	TanStack Query
Cliente GraphQL	graphql-request
Estilos	Tailwind CSS
Componentes UI	Radix UI Primitives
Formularios	React Hook Form + Zod
Iconos	Lucide React

TanStack Query (TanStack, 2024) gestiona la caché del servidor, las revalidaciones y los estados de carga/error, eliminando la necesidad de un gestor de estado global como Redux. **graphql-request** (Jamon Holmgren and contributors, 2024) actúa como cliente GraphQL ligero, adjuntando dinámicamente el encabezado `Authorization: Bearer <token>` en cada petición. **Tailwind CSS** (Tailwind Labs, 2024) combinado con **Radix UI** (WorkOS, 2024) proporciona componentes accesibles y un sistema de diseño consistente.

2.3. GraphQL frente a REST

La elección de GraphQL frente a REST responde a las características del dominio:

- Los libros tienen relaciones con autores, reseñas y préstamos. Una consulta típica necesita datos de varias entidades. Con REST serían necesarias 3–4 peticiones; con GraphQL, una sola.
- El catálogo de libros requiere filtros combinados (texto, género, disponibilidad, valoración mínima), que GraphQL gestiona mediante argumentos tipados.
- El esquema SDL actúa como documentación entre cliente y servidor, facilitando el desarrollo en paralelo.

3. Diseño del sistema

3.1. Arquitectura general

El sistema sigue una arquitectura de tres capas: cliente web (SPA), servidor GraphQL y base de datos.

El servidor expone un único endpoint `/graphql`. El contexto de Apollo (`buildContext`) se evalúa en cada petición: extrae el token del encabezado `Authorization`, lo verifica con `jsonwebtoken` y devuelve `{userId, userRole}`, que queda disponible para todos los resolvers.

3.2. Modelo de datos

El sistema define cinco colecciones en MongoDB. Las Tablas 3 a 7 describen los campos, tipos y restricciones de cada entidad.

Tabla 3
Entidad `Author`

Campo	Tipo	Requerido	Descripción
_id	ObjectId	Auto	Identificador primario
name	String	Sí	Nombre completo del autor
bio	String	No	Biografía
birthDate	Date	No	Fecha de nacimiento
deathDate	Date	No	Fecha de fallecimiento
nationality	String	No	Nacionalidad
createdAt	Date	Auto	Fecha de creación (timestamps)
updatedAt	Date	Auto	Última modificación (timestamps)

Tabla 4
Entidad `Book`

Campo	Tipo	Requerido	Descripción
_id	ObjectId	Auto	Identificador primario
title	String	Sí	Título del libro
isbn	String	No	Número ISBN
description	String	No	Descripción o sinopsis
publishedYear	Number	No	Año de publicación
genres	String[]	Sí (default [])	Lista de géneros
authorIds	ObjectId[]	Sí	Referencias a `Author`
averageRating	Number	Sí (default 0)	Valoración media recalculada
ratingsCount	Number	Sí (default 0)	Número de reseñas
availableCopies	Number	Sí (default 0)	Ejemplares disponibles
totalCopies	Number	Sí	Ejemplares totales

Tabla 5
Entidad `User`

Campo	Tipo	Requerido	Descripción
_id	ObjectId	Auto	Identificador primario
name	String	Sí	Nombre del usuario
email	String	Sí, único	Correo electrónico
passwordHash	String	Sí	Contraseña hasheada (bcrypt)
role	ADMIN | USER	Sí (default USER)	Rol del usuario
active	Boolean	Sí (default true)	Cuenta activa

Tabla 6
Entidad `Loan`

Campo	Tipo	Requerido	Descripción
_id	ObjectId	Auto	Identificador primario
userId	ObjectId (ref User)	Sí	Usuario que realiza el préstamo
bookId	ObjectId (ref Book)	Sí	Libro prestado
loanDate	Date	Sí (default now)	Fecha de inicio del préstamo
dueDate	Date	Sí	Fecha de devolución pactada
returnDate	Date	No	Fecha real de devolución
status	ACTIVE / RETURNED / LATE	Sí (default ACTIVE)	Estado del préstamo

Tabla 7
Entidad `Review`

Campo	Tipo	Requerido	Descripción
_id	ObjectId	Auto	Identificador primario
userId	ObjectId (ref User)	Sí	Usuario que escribe la reseña
bookId	ObjectId (ref Book)	Sí	Libro reseñado
rating	Number (1–5)	Sí	Valoración numérica
comment	String	No	Comentario de texto libre

La entidad `Review` define un índice compuesto único `{userId, bookId}` que garantiza como máximo una reseña por usuario y libro, con lo que se previene la duplicación de valoraciones. La entidad `Book` dispone de un índice de texto completo sobre `title` y `description` para soportar búsquedas por palabras clave.

3.3. Esquema GraphQL y operaciones disponibles

La Tabla 8 lista las principales operaciones del API.

Tabla 8
Operaciones GraphQL principales

Operación	Tipo	Descripción	Auth
`books`	Query	Catálogo con filtros y paginación	No
`book`	Query	Detalle de un libro por ID	No
`authors`	Query	Lista de autores con búsqueda	No
`author`	Query	Detalle de un autor por ID	No
`me`	Query	Datos del usuario autenticado	Sí
`myLoans`	Query	Préstamos del usuario actual	Sí
`allLoans`	Query	Todos los préstamos del sistema	ADMIN
`myReviews`	Query	Reseñas del usuario actual	Sí
`register`	Mutation	Crear cuenta de usuario	No
`login`	Mutation	Autenticar y obtener token JWT	No
`createBook`	Mutation	Añadir libro al catálogo	ADMIN
`updateBook`	Mutation	Modificar datos de un libro	ADMIN
`deleteBook`	Mutation	Eliminar un libro	ADMIN
`createAuthor`	Mutation	Añadir autor	ADMIN
`updateAuthor`	Mutation	Modificar datos de un autor	ADMIN
`deleteAuthor`	Mutation	Eliminar un autor	ADMIN
`createLoan`	Mutation	Registrar un nuevo préstamo	Sí
`returnLoan`	Mutation	Devolver un préstamo activo	Sí
`createReview`	Mutation	Publicar una reseña	Sí
`updateReview`	Mutation	Modificar una reseña propia	Sí
`deleteReview`	Mutation	Eliminar una reseña propia	Sí

Todas las queries que devuelven colecciones muestran la lista de elementos y un objeto `PageInfo` que incluye `totalItems`, `totalPages`, `hasNextPage` y `hasPreviousPage`. El límite máximo de resultados por página está fijado en 100.

3.4. Autenticación y autorización

El sistema implementa autenticación **sin estado** mediante JWT (Jones et al., 2015). El cliente envía la mutación `login`, recibe un token JWT (validez de 7 días) y lo almacena en `localStorage`. En cada petición posterior adjunta el encabezado `Authorization: Bearer <token>`, que `buildContext` verifica e inyecta como `userId` y `userRole` en el contexto de Apollo.

La autorización se implementa con dos guardas definidas en `src/utils/requireAuth.ts`. La guarda `requireAuth(ctx)` lanza `UNAUTHENTICATED` si no hay sesión activa; la guarda `requireAdmin(ctx)` lanza `FORBIDDEN` si el rol no es `ADMIN`. Ambas funciones se invocan al inicio de cualquier resolver protegido.

4. Implementación

4.1. Backend

4.1.1. Punto de entrada y configuración del servidor

El fichero `src/index.ts` orquesta el arranque: conecta a MongoDB mediante Mongoose, instancia `ApolloServer<Context>` con los resolvers raíz y el esquema SDL, y monta el servidor en el endpoint `/graphql` mediante `expressMiddleware`. En el arranque también se invoca `checkLateLoans()` para marcar como `LATE` todos los préstamos activos cuya fecha de devolución haya vencido.

4.1.2. Organización de resolvers

Los resolvers se dividen en ficheros por entidad bajo `src/resolvers/`. Cada fichero exporta un objeto con las claves `Query`, `Mutation` y el nombre del tipo (p. ej., `Book`). El fichero raíz `src/resolvers/index.ts` los fusiona mediante *spread* de objetos, por lo que añadir una nueva entidad solo requiere crear su fichero y extender el spread.

4.1.3. Utilidades del servidor

La Tabla 9 describe las utilidades más relevantes del servidor.

Tabla 9
Utilidades del servidor (`src/utils/`)

Función	Propósito
`paginate(model, filter)`	Paginación genérica sobre cualquier modelo Mongoose; cap de 100
`recalculateBookRating`	Recalcula `averageRating` y `ratingsCount` vía agregación
`checkLateLoans`	Marca como LATE los préstamos activos vencidos (bulk update)
`assertValidId`	Valida el formato ObjectId antes de consultar la BD
`handleMongooseError`	Convierte `ValidationError` de Mongoose en `GraphQLError`
`requireAuth / requireAdmin`	Guardas de autenticación/autorización en resolvers

4.1.4. Proceso de préstamo

La mutación `createLoan` verifica la sesión activa, valida el formato del identificador del libro, comprueba que existan copias disponibles y que la fecha de devolución sea futura, crea el documento `Loan` con estado `ACTIVE` y decrementa `availableCopies` en una unidad. La mutación `returnLoan` verifica la propiedad del préstamo, comprueba que su estado sea `ACTIVE`, lo marca como `RETURNED` con la fecha de devolución real e incrementa de nuevo `availableCopies`.

4.2. Frontend

4.2.1. Estructura de páginas

La Tabla 10 enumera las páginas de la aplicación cliente.

Tabla 10
Páginas del cliente React

Ruta	Componente	Descripción	Auth
`/`	Books	Catálogo con búsqueda y filtros	No
`/books/:id`	BookDetail	Detalle, reseñas y acción de préstamo	No
`/login`	Login	Formulario de inicio de sesión	No
`/register`	Register	Formulario de registro	No
`/my-loans`	MyLoans	Préstamos del usuario, opción devolver	Sí
`/my-reviews`	MyReviews	Reseñas del usuario, editar/eliminar	Sí
`/admin/books`	AdminBooks	CRUD completo de libros	ADMIN
`/admin/authors`	AdminAuthors	CRUD completo de autores	ADMIN
`/admin/loans`	AdminLoans	Vista global de préstamos con filtros	ADMIN

4.2.2. Panel de administración (`/admin`)

La sección `/admin` agrupa las tres vistas de gestión exclusivas del rol `ADMIN`, accesibles mediante una barra de navegación interna con pestañas (*Libros*, *Autores*, *Préstamos*). El acceso está protegido por el componente `AdminRoute`, que redirige al catálogo si el usuario no tiene rol `ADMIN`.

- **AdminBooks** (`/admin/books`): tabla paginada con búsqueda por título. Permite crear un libro nuevo (botón *Nuevo libro*), editarlo (icono lápiz) o eliminarlo (icono papelera). Tanto la creación como la edición se realizan mediante un diálogo modal con campos para título, ISBN, descripción, año de publicación, géneros (separados por coma), autores y número de copias totales y disponibles.
- **AdminAuthors** (`/admin/authors`): tabla con los autores del catálogo. Permite crear, editar y eliminar autores a través de un diálogo modal con los campos nombre, biografía, nacionalidad, fecha de nacimiento y fecha de fallecimiento.
- **AdminLoans** (`/admin/loans`): vista de solo lectura de todos los préstamos del sistema, con filtro por estado (*ACTIVE*, *RETURNED*, *LATE*) y paginación. Utiliza la query `allLoans` restringida a `ADMIN`.

Los hooks de datos del panel se centralizan en `client/src/hooks/useAdmin.ts`. Expone hooks de consulta (`useAuthors`, `useAllLoans`) y hooks de mutación:

- Libros: `useCreateBook`, `useUpdateBook`, `useDeleteBook`.
- Autores: `useCreateAuthor`, `useUpdateAuthor`, `useDeleteAuthor`.

Cada mutación invalida las queries correspondientes para mantener la caché de TanStack Query sincronizada sin recargar la página.

4.2.3. Patrón de hooks

Cada dominio de datos cuenta con su propio fichero de hooks en `client/src/hooks/`. Todos siguen el mismo patrón: usan `useQuery` o `useMutation` de TanStack Query, que internamente llama a `gqlClient.request()` con la operación GraphQL correspondiente. Las mutations invalidan las queries relacionadas tras ejecutarse con éxito, manteniendo la caché sincronizada sin necesidad de actualización manual.

El cliente GraphQL es una instancia única de `graphql-request` definida en `client/src/lib/graphql.ts`. Lee el token del `localStorage` en cada petición, garantizando que las cabeceras de autorización siempre estén actualizadas.

5. Manual de usuario

5.1. Registro e inicio de sesión

Para acceder a las funcionalidades de préstamo y reseñas es necesario disponer de una cuenta:

1. Navegar a **/register**. Rellenar los campos *Nombre*, *Correo electrónico* y *Contraseña* (mínimo 6 caracteres).
2. Hacer clic en *Crear cuenta*. Si el registro es correcto, el sistema inicia sesión automáticamente y redirige al catálogo.
3. Para iniciar sesión en una sesión posterior, navegar a **/login**, introducir correo y contraseña, y hacer clic en *Entrar*.

5.2. Exploración del catálogo

La página principal (/) muestra el catálogo de libros en una cuadrícula paginada con 9 libros por página. Las opciones de filtrado disponibles son:

- **Búsqueda por texto**: campo de búsqueda en la parte superior.
- **Género**: menú desplegable con los géneros disponibles.
- **Solo disponibles**: interruptor que oculta los libros sin ejemplares disponibles.
- **Paginación**: botones de página en la parte inferior del catálogo.

Hacer clic en cualquier tarjeta de libro lleva al detalle del libro (**/books/:id**), donde se muestra la descripción completa, la valoración media, los autores y las reseñas de otros usuarios.

La Figura 1 muestra la vista principal del catálogo con los filtros disponibles.

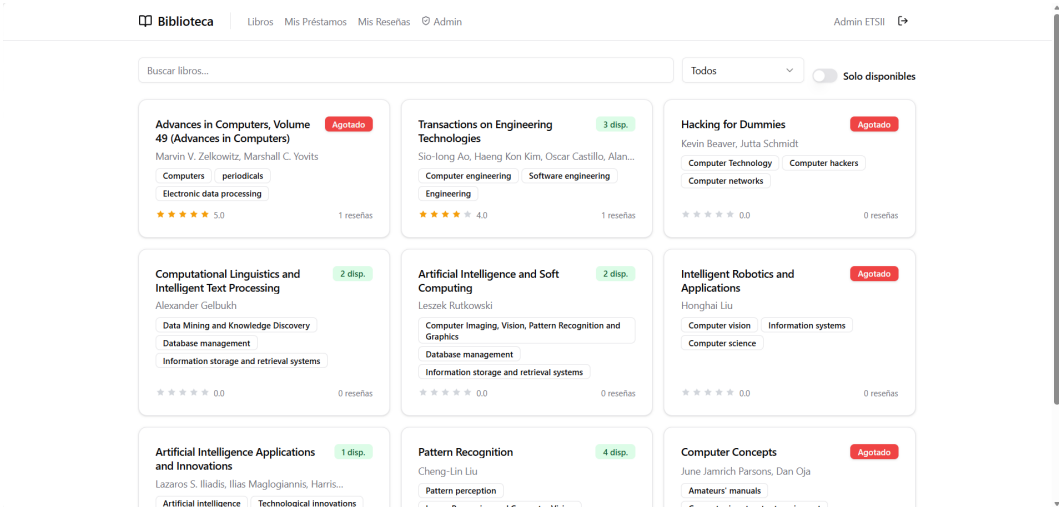

Figura 1. Catálogo de libros con búsqueda y filtros

5.3. Gestión de préstamos

1. En la página de detalle de un libro, si hay ejemplares disponibles y el usuario está autenticado, aparece el botón *Solicitar préstamo*.
2. Al hacer clic se abre un diálogo donde se selecciona la fecha de devolución (*dueDate*). La fecha debe ser posterior a la actual. La Figura 2 muestra esta pantalla.
3. Confirmar el préstamo. El contador de ejemplares disponibles se actualiza automáticamente.
4. Para ver los préstamos activos, navegar a **/my-loans**. Desde allí se puede devolver cualquier préstamo con estado *ACTIVO* haciendo clic en *Devolver*.
5. Los préstamos cuya fecha de devolución ha vencido aparecen con estado *TARDE*.

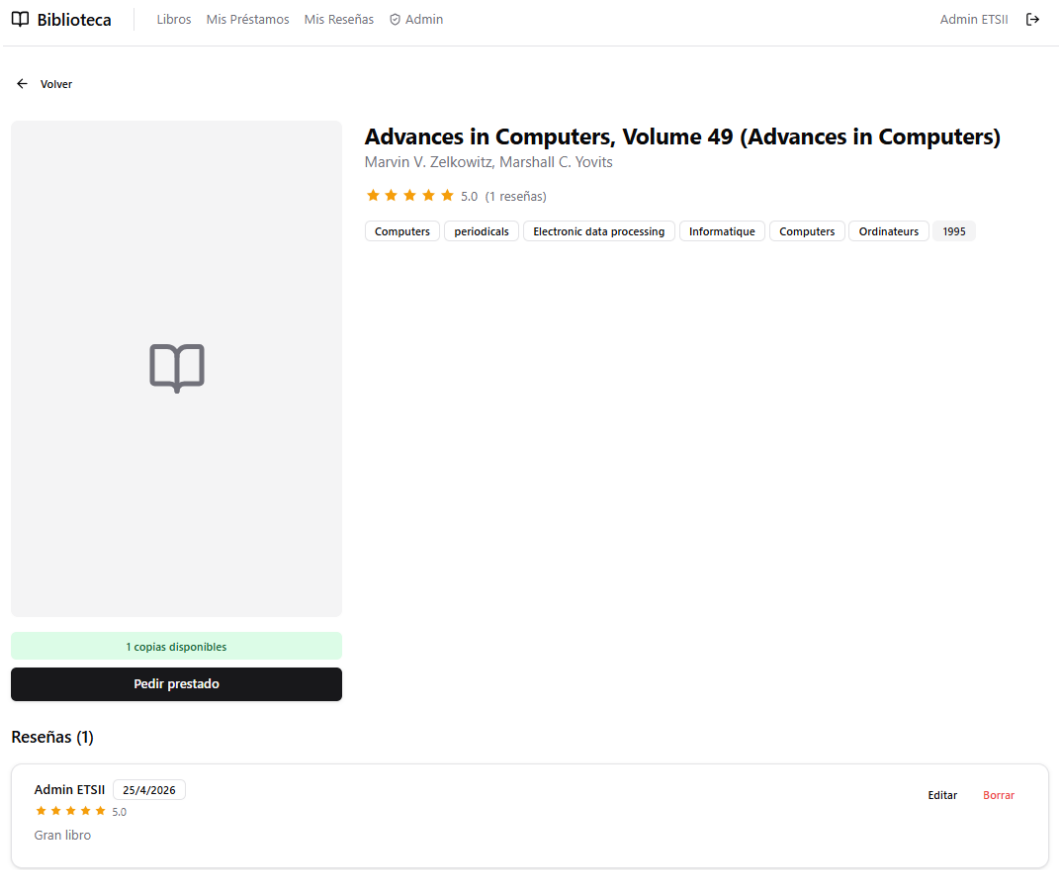

Figura 2. Página de detalle de libro con botón de préstamo

La Figura 3 muestra la vista *Mis préstamos* con los préstamos activos del usuario.

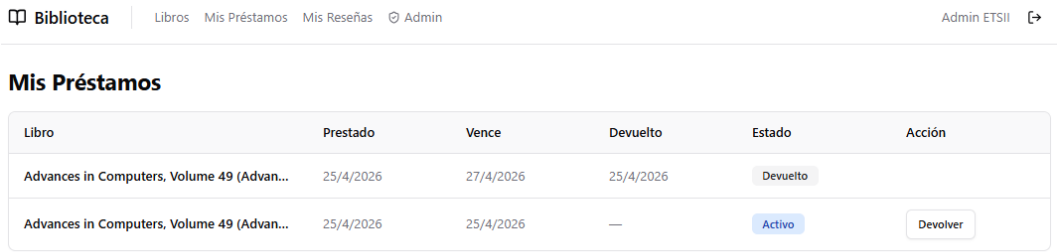

Figura 3. Vista de préstamos del usuario con opción de devolución

5.4. Gestión de reseñas

1. En la página de detalle del libro, usuarios autenticados pueden publicar una reseña mediante el formulario situado bajo la lista de reseñas existentes.
2. Se asigna una puntuación de 1 a 5 estrellas y, opcionalmente, un comentario de texto libre.
3. Solo se permite **una reseña por usuario y libro**. El sistema rechaza duplicados.
4. Las reseñas propias pueden eliminarse desde la misma página de detalle o desde **/my-reviews**. Desde */my-reviews* también es posible modificar la puntuación o el comentario.

La Figura 4 muestra la página *Mis reseñas* con las opciones de edición y eliminación disponibles para el usuario autenticado.

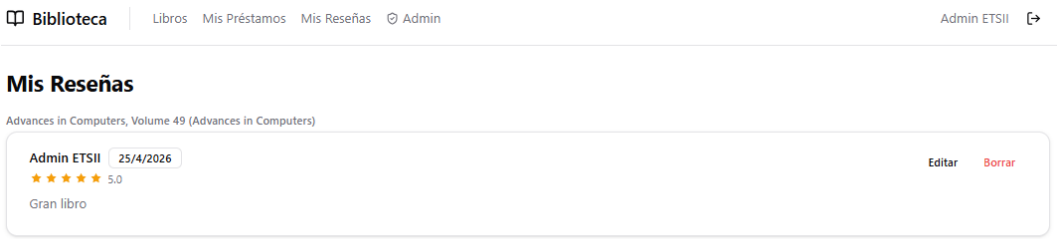

Figura 4. Vista de reseñas del usuario con opciones de edición y eliminación

5.5. Funciones de administrador

Los usuarios con rol `ADMIN` tienen acceso al panel de administración en **/admin**, accesible desde el enlace *Admin* en la barra de navegación superior (solo visible para administradores). El panel está organizado en tres pestañas:

- **Libros** (`/admin/books`): tabla con todos los libros del catálogo. El botón *Nuevo libro* abre un formulario modal para añadir un libro. Cada fila dispone de iconos para editar o eliminar el libro.
- **Autores** (`/admin/authors`): gestión análoga para los autores, con formulario modal de creación y edición.
- **Préstamos** (`/admin/loans`): listado global de todos los préstamos del sistema con filtro desplegable por estado (*Activo*, *Devuelto*, *Tardío*) y paginación.

La Figura 5 muestra el panel de gestión de libros con la tabla y los controles de creación y edición. Las Figuras 6 y 7 muestran, respectivamente, la gestión de autores y la vista global de préstamos.

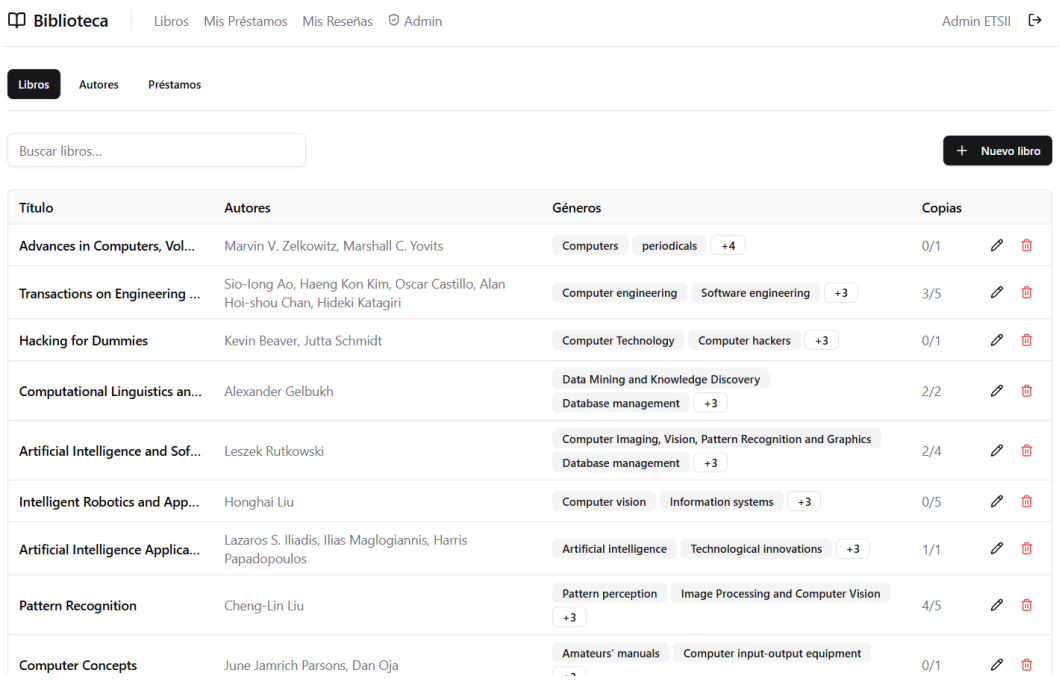

Figura 5. Panel de administración — gestión de libros (`/admin/books`)

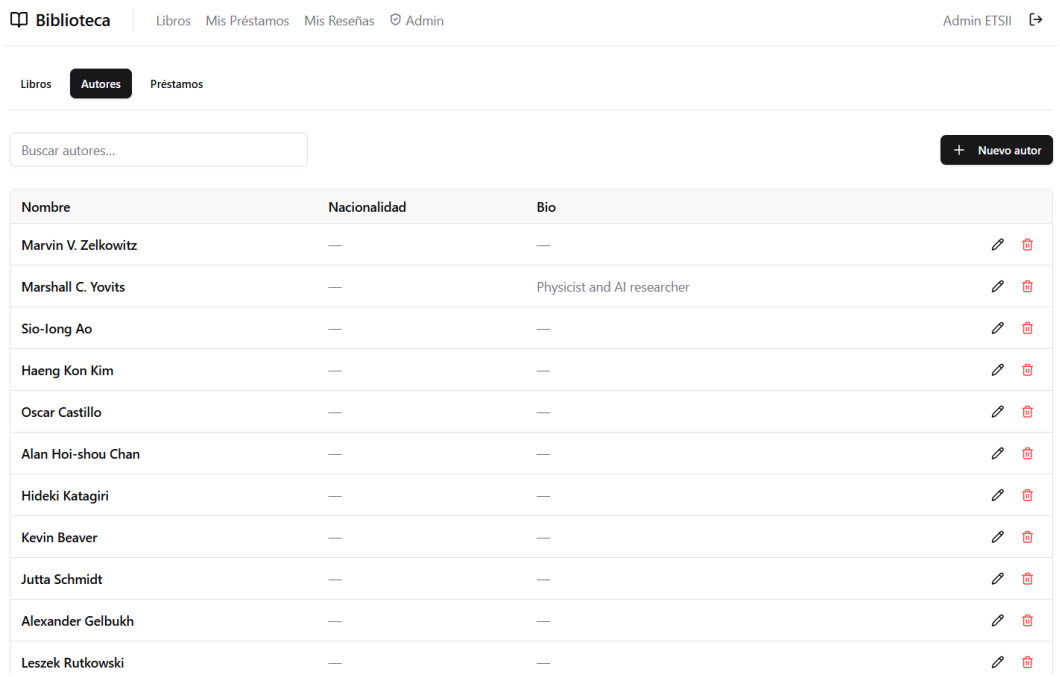

Figura 6. Panel de administración — gestión de autores (`/admin/authors`)

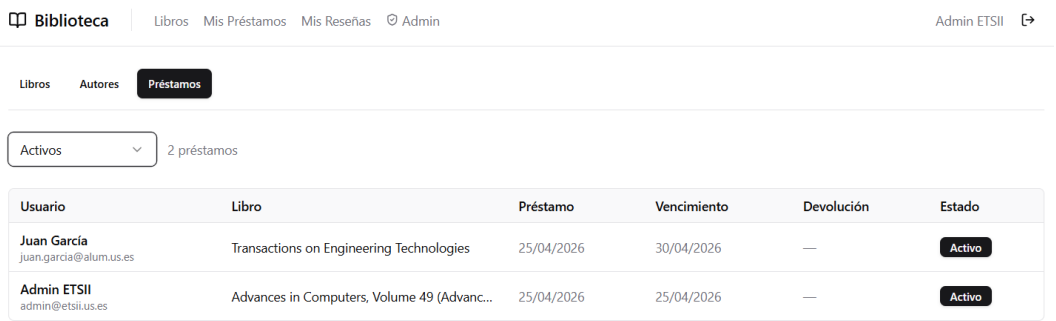

Figura 7. Panel de administración — vista global de préstamos (`/admin/loans`)

Adicionalmente, todas estas operaciones están disponibles desde el Apollo Sandbox (`http://localhost:4000/graphql`) para usuarios que prefieran interactuar directamente con la API.

Para crear un usuario administrador, tras registrarse con el flujo normal, un desarrollador debe actualizar su rol directamente en MongoDB:

```
db.users.updateOne({ email: "admin@biblioteca.com" }, { $set: {
    role: "ADMIN" } })
```

6. Manual de despliegue

6.1. Requisitos previos

- **Node.js 18 o superior**: disponible en https://nodejs.org.
- **npm 9+** (incluido con Node.js).
- **Git** (para clonar el repositorio).

6.2. Configuración del entorno

El servidor requiere un fichero `.env` en la raíz del repositorio con las variables listadas en la Tabla 11.

Tabla 11
Variables de entorno del servidor

Variable	Descripción	Obligatoria
`MONGODB_URI`	URI de conexión a MongoDB (p. ej. `mongodb://localhost/biblioteca`)	Sí
`JWT_SECRET`	Clave secreta para firmar y verificar JWT	Sí
`PORT`	Puerto HTTP del servidor (defecto: 4000)	No

Copiar el fichero de ejemplo y editarlo:

```
cp .env.example .env
# Editar .env con el editor preferido
```

6.3. Instalación y arranque

6.3.1. Backend

```
# Clonar el repositorio
git clone https://github.com/angelmanuelferrer/Biblioteca-
   Digital-CBD.git
cd Biblioteca-Digital-CBD

# Instalar dependencias del servidor
npm install

# Arrancar en modo desarrollo (hot reload con nodemon)
npm run dev

# El servidor queda disponible en http://localhost:4000/graphql
```

Para producción, compilar primero el TypeScript y luego ejecutar el resultado:

```
npm run build   # genera la carpeta dist/
npm start       # ejecuta node dist/index.js
```

6.3.2. Cliente web

```
cd client
npm install
npm run dev     # Vite dev server en http://localhost:5173
```

El cliente asume que el servidor escucha en `http://localhost:4000`. Si se cambia el puerto, actualizar la constante en `client/src/lib/graphql.ts`.

6.4. Aplicación desplegada

La aplicación se encuentra desplegada en la plataforma **Render** y es accesible públicamente en la siguiente dirección:

https://biblioteca-digital-cbd.onrender.com/

El servidor GraphQL y el cliente web se sirven desde el mismo dominio. El endpoint de la API es `https://biblioteca-digital-cbd.onrender.com/graphql` y acepta peticiones POST con el cuerpo en formato JSON. El Apollo Sandbox solo está disponible en entorno de desarrollo local (`http://localhost:4000/graphql`), ya que Apollo Server 4 lo deshabilita automáticamente cuando `NODE_ENV=production`.

Nota: al estar desplegado en el plan gratuito de Render, el servidor puede tardar unos segundos en responder si lleva inactivo un tiempo (*cold start*).

6.5. Población de datos de prueba

El script de semilla descarga aproximadamente 200 libros de ciencias de la computación desde la Open Library API (Internet Archive, 2024), crea los autores correspondientes y genera diez usuarios de prueba:

```
npm run seed
```

La Tabla 12 lista las credenciales de acceso creadas por el script.

Tabla 12
Usuarios de prueba creados por el script de semilla

Correo electrónico	Contraseña	Rol
`admin@etsii.us.es`	Admin1234!	ADMIN
`juan.garcia@alum.us.es`	User1234!	USER
`maria.lopez@alum.us.es`	User1234!	USER
`carlos.martinez@alum.us.es`	User1234!	USER
`ana.fernandez@alum.us.es`	User1234!	USER
`luis.perez@alum.us.es`	User1234!	USER
`sofia.ruiz@alum.us.es`	User1234!	USER
`david.torres@alum.us.es`	User1234!	USER
`elena.sanchez@alum.us.es`	User1234!	USER
`pablo.jimenez@alum.us.es`	User1234!	USER

Nota: el script de semilla elimina previamente los datos existentes en las colecciones `authors`, `books` y `users`. No debe ejecutarse en entornos de producción.

7. Conclusiones

7.1. Resumen de resultados

El proyecto ha confirmado que GraphQL, combinado con Apollo Server 4 y MongoDB, constituye un stack maduro y productivo para aplicaciones de gestión de datos relacionados. La capacidad de resolver en una sola petición el detalle de un libro junto con sus autores, reseñas y disponibilidad simplificó sustancialmente la lógica del cliente y eliminó las múltiples idas y venidas que habría requerido una API REST equivalente.

El esquema SDL sirvió como contrato explícito entre frontend y backend desde el inicio del desarrollo, lo que permitió avanzar en ambas capas con seguridad de tipos de extremo a extremo. El resultado es un sistema completo, con autenticación por roles, paginación genérica y un modelo de datos con restricciones de integridad que cubre todos los flujos del dominio bibliotecario.

7.2. Dificultades y decisiones de diseño

Entre los retos encontrados cabe destacar la sincronización concurrente del contador `availableCopies`, resuelta delegando la actualización en la capa de resolvers mediante operaciones atómicas de Mongoose. La recalculación de la valoración media (`recalculateBookRating`) se implementó mediante una *pipeline* de agregación de MongoDB, lo que garantiza precisión independientemente del número de reseñas.

Referencias

Apollo GraphQL (2023). *Apollo Server 4 Documentation*. Último acceso: 14 de abril de 2026.

Automattic, Inc. (2024). *Mongoose Documentation v8*. Último acceso: 14 de abril de 2026.

GraphQL Foundation (2021). *GraphQL Specification (October 2021 Edition)*. Último acceso: 14 de abril de 2026.

Internet Archive (2024). *Open Library API*. Último acceso: 14 de abril de 2026.

Jamon Holmgren and contributors (2024). *graphql-request — Minimal GraphQL Client*. Último acceso: 14 de abril de 2026.

Jones, M., Bradley, J., and Sakimura, N. (2015). RFC 7519: JSON Web Token (JWT). Internet Engineering Task Force (IETF). Último acceso: 14 de abril de 2026.

Tailwind Labs (2024). *Tailwind CSS Documentation v3*. Último acceso: 14 de abril de 2026.

TanStack (2024). *TanStack Query v5 Documentation*. Último acceso: 14 de abril de 2026.

WorkOS (2024). *Radix UI Primitives*. Último acceso: 14 de abril de 2026.

Uso de IA

En el desarrollo de este proyecto se ha empleado **Claude Code** (Anthropic, modelo claude-sonnet-4-6) como asistente de programación. A continuación se detalla en qué partes del trabajo se ha utilizado y con qué propósito:

- **Generación de código scaffolding**: se utilizó Claude Code para generar la estructura inicial de los resolvers de las entidades *Author*, *Book*, *Loan* y *Review*, así como las utilidades del servidor (`paginate`, `recalculateBookRating`, `checkLateLoans`). El código generado fue revisado, adaptado y comprendido por los autores antes de incorporarlo al repositorio.
- **Script de semilla**: Claude Code asistió en el diseño del script `scripts/seed.ts` para consumir la Open Library API con control de límites de peticiones (*rate limiting*).
- **Documentación**: la estructura y el contenido de esta memoria se elaboraron con apoyo de Claude Code, que proporcionó sugerencias de organización y redacción en español.

En todas las secciones donde se utilizó asistencia de IA, los autores han comprobado y validado la información generada, añadiendo las referencias bibliográficas correspondientes. Todo el contenido entregado es comprendido y puede ser explicado y defendido por los autores, tal como exigen las instrucciones de la asignatura.